

Unit: 5 Memory management

Prepared by:
Er. Binod Kumar Rajbhar

Memory Management

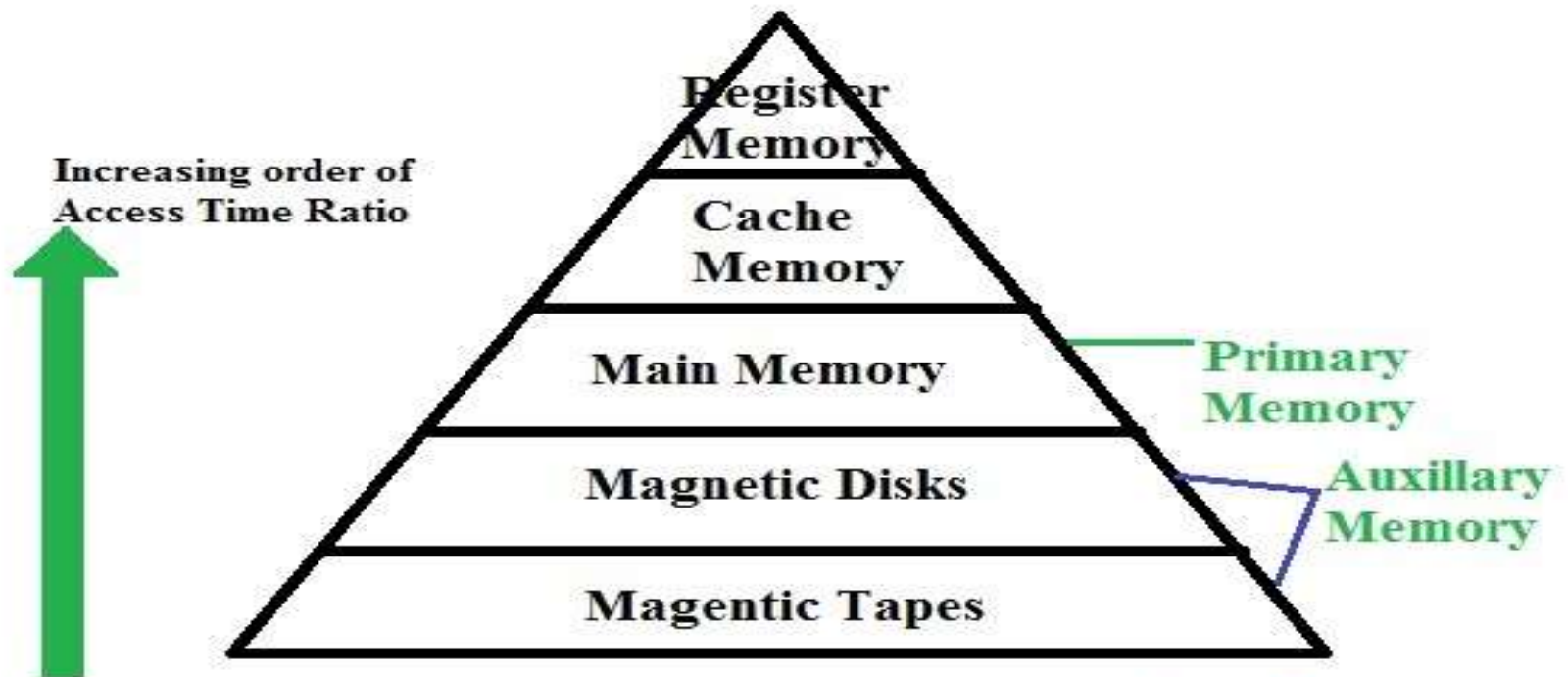
- ✓ Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution.
- ✓ Memory management keeps track of each and every memory location, regardless of either it is allocated to some process or it is free.
- ✓ It checks how much memory is to be allocated to processes. It decides which process will get memory at what time.
- ✓ It tracks whenever some memory gets freed or unallocated and correspondingly it updates the status.

What is Memory Management?

- ❖ **Memory Management** is the process of controlling and coordinating computer memory, assigning portions known as blocks to various running programs to optimize the overall performance of the system.
- ❖ It is the most important function of an operating system that manages primary memory.
- ❖ It helps processes to move back and forward between the main memory and execution disk.
- ❖ It helps OS to keep track of every memory location, irrespective of whether it is allocated to some process or it remains free.

- **What is Memory Hierarchy?**

- ❖ The memory in a computer can be divided into five hierarchies based on the speed as well as use.
- ❖ The processor can move from one level to another based on its requirements.
- ❖ The five hierarchies in the memory are registers, cache, main memory, magnetic discs, and magnetic tapes.
- ❖ The first three hierarchies are volatile memories which mean when there is no power, and then automatically they lose their stored data.
- ❖ Whereas the last two hierarchies are not volatile which means they store the data permanently.
- ❖ The **memory hierarchy design** in a computer system mainly includes different storage devices.
- ❖ Most of the computers were inbuilt with extra storage to run more powerfully beyond the main memory capacity.
- ❖ The following **memory hierarchy diagram** is a hierarchical pyramid for computer memory.
- ❖ The designing of the memory hierarchy is divided into two types such as primary (Internal) memory and secondary (External) memory.



©www.Elprocus.com

Characteristics of Memory Hierarchy

- **Performance**

- ❖ Previously, the designing of a computer system was done without memory hierarchy, and the speed gap among the main memory as well as the CPU registers enhances because of the huge disparity in access time, which will cause the lower performance of the system.
- ❖ So, the enhancement was mandatory. The enhancement of this was designed in the memory hierarchy model due to the system's performance increase.

- **Ability**

- ❖ The ability of the memory hierarchy is the total amount of data the memory can store.
- ❖ Because whenever we shift from top to bottom inside the memory hierarchy, then the capacity will increase.

- **Access Time**

- ❖ The access time in the memory hierarchy is the interval of the time among the data availability as well as request to read or write.
- ❖ Because whenever we shift from top to bottom inside the memory hierarchy, then the access time will increase

- **Cost per bit**

- ❖ When we shift from bottom to top inside the memory hierarchy, then the cost for each bit will increase which means an internal Memory is expensive compared with external memory.

Logical and Physical Address

- ✓ In operating systems, logical and physical addresses are used to manage and access memory.
- ✓ Here's an overview of each:

❖ Logical address:

- ✓ A logical address, also known as a virtual address, is an address generated by the CPU during program execution.
- ✓ It is the address seen by the process and is relative to the program's address space.
- ✓ The process accesses memory using logical addresses, which are translated by the operating system into physical addresses.

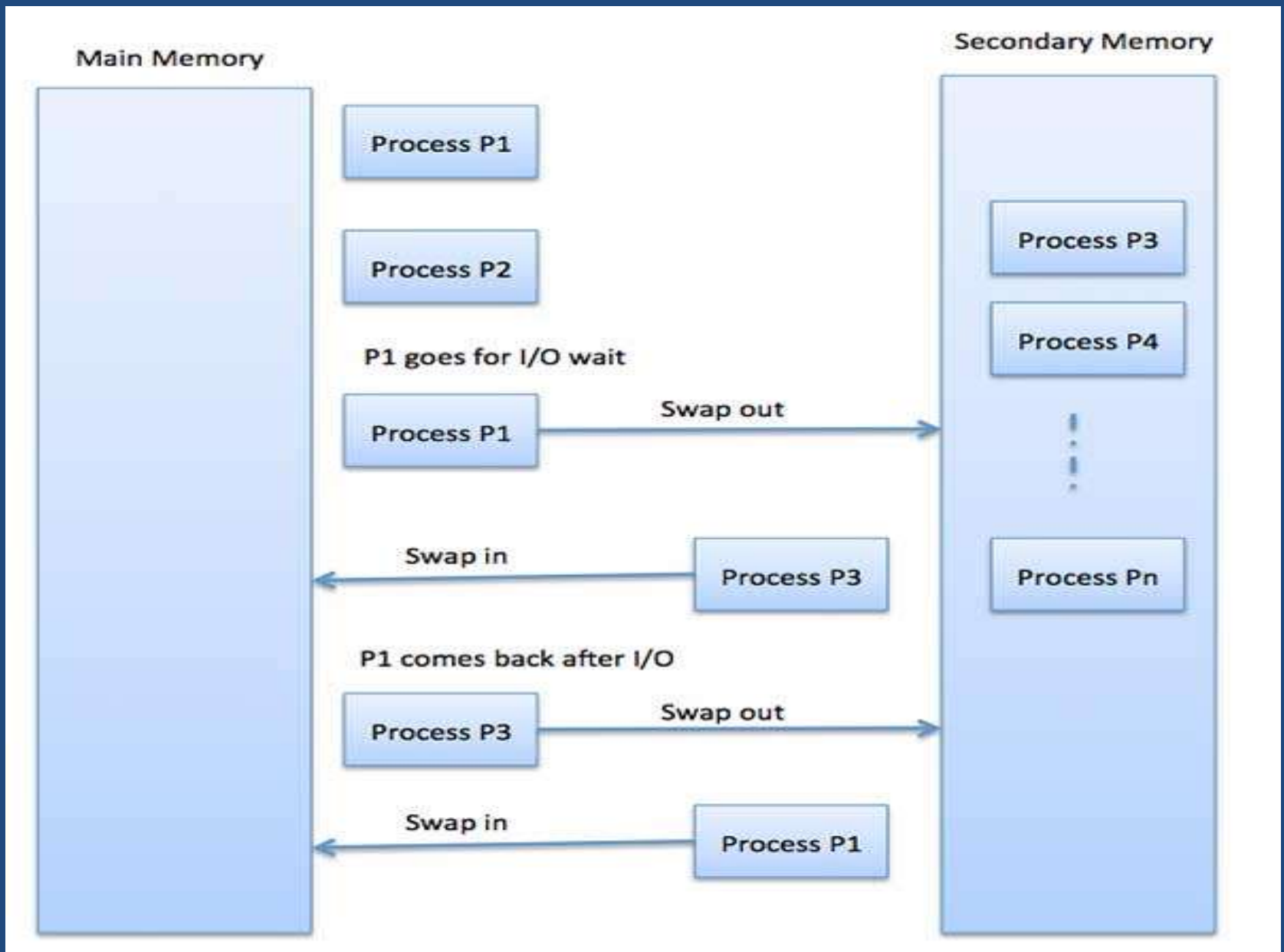
Physical address:

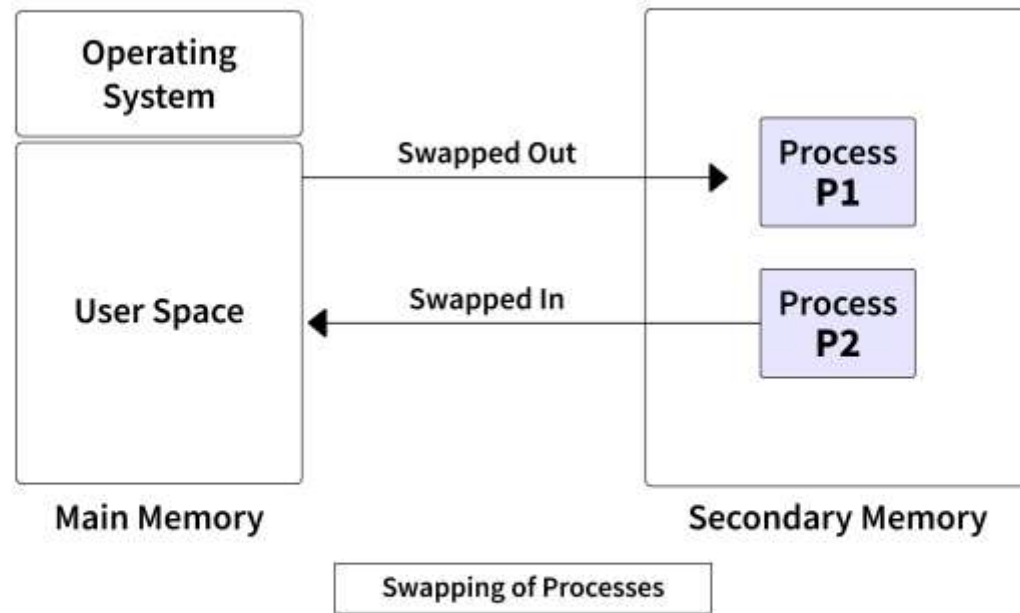
- ✓ A physical address is the actual address in main memory where data is stored.
- ✓ It is a location in physical memory, as opposed to a virtual address.
- ✓ Physical addresses are used by the memory management unit (MMU) to translate logical addresses into physical addresses.
- ✓ The translation from logical to physical addresses is performed by the operating system's memory management unit.
- ✓ The MMU uses a page table to translate logical addresses into physical addresses.
- ✓ The page table maps each logical page number to a physical frame number.

Memory Management with Swapping

Swapping:

- ✓ Swapping is a mechanism in which a process can be swapped temporarily out of main memory (or move) to secondary storage (disk) and make that memory available to other processes.
- ✓ At some later time, the system swaps back the process from the secondary storage to main memory.
- ✓ **Swapping in OS** is one of those schemes which fulfill the goal of maximum utilization of **CPU** and memory management by swapping in and swapping out processes from the main memory.
- ✓ Swap in removes the process from **hard drive**(secondary memory) and swap out removes the process from **RAM**(main memory).





SCALER
Topics

Swap In:

- The method of removing a process from secondary memory (**Hard Drive**) and restoring it to the main memory (**RAM**) for execution is known as the Swap In method.

Swap Out:

- It is a method of bringing out a process from the main memory(**RAM**) and sending it to the secondary memory(**hard drive**) so that the processes with higher priority or more memory consumption will be executed is known as the Swap Out method.

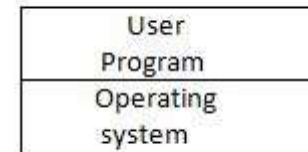
Advantages

- Swapping in OS helps in achieving the goal of Maximum CPU Utilization.
- Swapping ensures proper memory availability for every process that needs to be executed.
- Swapping helps in avoiding the problem of process starvation means a process should not take much time for execution so that the next process should be executed.
- CPU can perform various tasks simultaneously with the help of swapping so that processes do not have to wait much longer before execution.
- Swapping ensures proper RAM(main memory) utilization.

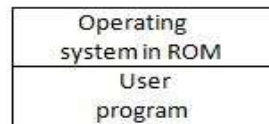
Basic Memory Management Techniques

Mono programming without swapping or paging:-

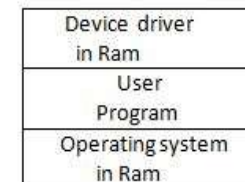
- ❖ This is the simplest memory management in which just one program is run at a time, sharing the memory between the program and the operating system.
- ❖ Three variables are used in this scheme.
 1. The operating system may be at the bottom of memory in ram.



- Memory management
- 2. The operating system may be in ROM at top of memory.



- Memory management
- 3. Device driver may be at the top of the RAM.



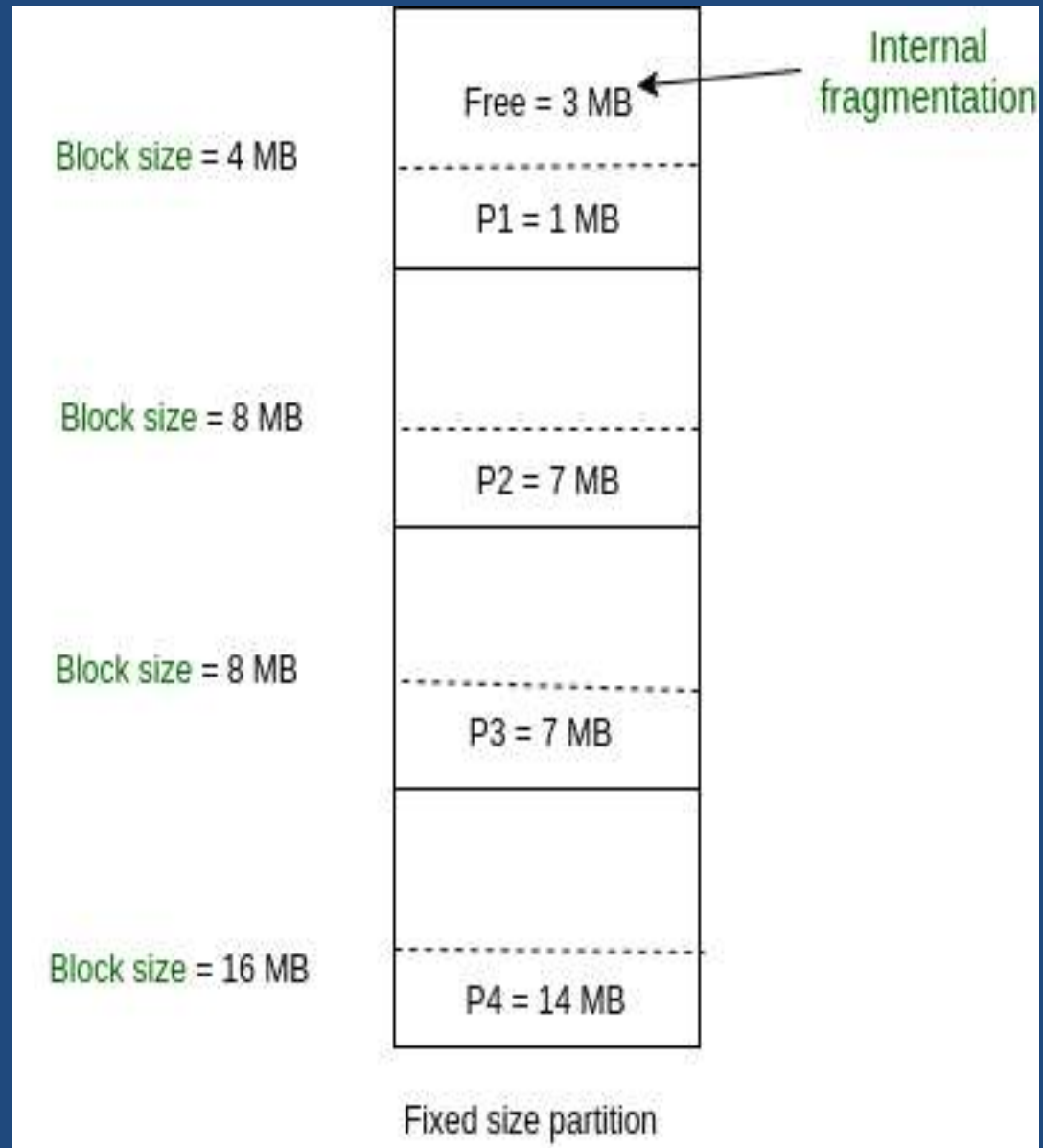
Memory management in multiprogramming

Multiprogramming with fixed partition:

- ❖ There are two Memory Management Techniques: **Contiguous**, and **Non-Contiguous**.
- ❖ In Contiguous Technique, executing process must be loaded entirely in main-memory.
- ❖ Contiguous Technique can be divided into:
 - Fixed (or static) partitioning
 - Variable (or dynamic) partitioning

Fixed Partitioning:

- ❖ This is the oldest and simplest technique used to put more than one processes in the main memory.
- ❖ In this partitioning, number of partitions (non-overlapping) in RAM are **fixed but size** of each partition may or **may not be same**.
- ❖ As it is **contiguous** allocation, hence no spanning is allowed.
- ❖ Here partition are made before execution or during system configure.



❖ As illustrated in above figure, first process is only consuming 1MB out of 4MB in the main memory. Hence, Internal Fragmentation in first block is $(4-1) = 3\text{MB}$.

Sum of Internal Fragmentation in every block = $(4-1)+(8-7)+(8-7)+(16-14) = 3+1+1+2 = 7\text{MB}$.

❖ Suppose process P5 of size 7MB comes.

❖ But this process cannot be accommodated inspite of available free space because of contiguous allocation (as spanning is not allowed).

❖ Hence, 7MB becomes part of External Fragmentation.

Advantages of Fixed Partitioning –

- **Easy to implement:**

Algorithms needed to implement Fixed Partitioning are easy to implement. It simply requires putting a process into certain partition without focussing on the emergence of Internal and External Fragmentation.

- **Little OS overhead:**

Processing of Fixed Partitioning require lesser excess and indirect computational power

Disadvantages of Fixed Partitioning –

- **Internal Fragmentation:**

Main memory use is inefficient. Any program, no matter how small, occupies an entire partition. This can cause internal fragmentation.

- **External Fragmentation:**

The total unused space (as stated above) of various partitions cannot be used to load the processes even though there is space available but not in the contiguous form (as spanning is not allowed).

- **Limit process size:**

Process of size greater than size of partition in Main Memory cannot be accommodated. Partition size cannot be varied according to the size of incoming process's size. Hence, process size of 32MB in above stated example is invalid.

- **Limitation on Degree of Multiprogramming:**

Partition in Main Memory are made before execution or during system configure. Main Memory is divided into fixed number of partition. Suppose if there are n partitions in RAM and are the number of processes, then condition must be fulfilled. Number of processes greater than number of partitions in RAM is invalid in Fixed Partitioning.

Variable Partitioning

- ❖ It is a part of Contiguous allocation technique.
- ❖ It is used to alleviate the problem faced by Fixed Partitioning.
- ❖ In contrast with fixed partitioning, partitions are not made before the execution or during system configure.
- ❖ Various **features** associated with variable Partitioning-
 - Initially RAM is empty and partitions are made during the run-time according to process's need instead of partitioning during system configure.
 - The size of partition will be equal to incoming process.
 - The partition size varies according to the need of the process so that the internal fragmentation can be avoided to ensure efficient utilization of RAM.
 - Number of partitions in RAM is not fixed and depends on the number of incoming process and Main Memory's size.

Dynamic partitioning

Operating system
P1 = 2 MB
P2 = 7 MB
P3 = 1 MB
P4 = 5 MB
Empty space of RAM

Block size = 2 MB

Block size = 7 MB

Block size = 1 MB

Block size = 5 MB

Partition size = process size
So, no internal Fragmentation

Advantages of Variable Partitioning –

- **No Internal Fragmentation:**

In variable Partitioning, space in main memory is allocated strictly according to the need of process, hence there is no case of internal fragmentation. There will be no unused space left in the partition.

- **No restriction on Degree of Multiprogramming:**

More number of processes can be accommodated due to absence of internal fragmentation. A process can be loaded until the memory is empty.

- **No Limitation on the size of the process:**

In Fixed partitioning, the process with the size greater than the size of the largest partition could not be loaded and process can not be divided as it is invalid in contiguous allocation technique. Here, In variable partitioning, the process size can't be restricted since the partition size is decided according to the process size.

Disadvantages of Variable Partitioning –

- **Difficult Implementation:**

Implementing variable Partitioning is difficult as compared to Fixed Partitioning as it involves allocation of memory during run-time rather than during system configure.

- **External Fragmentation:**

There will be external fragmentation inspite of absence of internal fragmentation.

- For example, suppose in above example- process P1(2MB) and process P3(1MB) completed their execution. Hence two spaces are left i.e. 2MB and 1MB. Let's suppose process P5 of size 3MB comes. The empty space in memory cannot be allocated as no spanning is allowed in contiguous allocation. The rule says that process must be contiguously present in main memory to get executed. Hence it results in External Fragmentation.

Dynamic partitioning

Operating system
P1 (2 MB) executed, now empty
P2 = 7 MB
P3 (1 MB) executed
P4 = 5 MB
Empty space of RAM

Block size = 2 MB

Block size = 7 MB

Block size = 1 MB

Block size = 5 MB

Partition size = process size
So, no internal Fragmentation

Now P5 of size 3 MB cannot be accommodated in spite of required available space because in contiguous no spanning is allowed.

Difference between Fixed Partitioning and Variable Partitioning:

S.NO.	Fixed partitioning	Variable partitioning
1.	❖ In multi-programming with fixed partitioning the main memory is divided into fixed sized partitions.	❖ In multi-programming with variable partitioning the main memory is not divided into fixed sized partitions.
2.	❖ Only one process can be placed in a partition.	❖ In variable partitioning, the process is allocated a chunk of free memory.
3.	❖ It does not utilize the main memory effectively.	❖ It utilizes the main memory effectively.
4.	❖ There is presence of internal fragmentation and external fragmentation.	❖ There is external fragmentation.
5.	❖ Degree of multi-programming is less.	❖ Degree of multi-programming is higher.
6.	❖ It is easier to implement.	❖ It is less easy to implement.
7.	❖ There is limitation on size of process.	❖ There is no limitation on size of process.

Relocation and Protection

- **Relocation**: When a program is run it does not know in advance what location it will be loaded at. Therefore, the program cannot simply generate static addresses (e.g. from jump instructions). Instead, they must be made relative to where the program has been loaded.
- **Protection**: Once you can have two programs in memory at the same time there is a danger that one program can write to the address space of another program. This is obviously dangerous and should be avoided.

Compaction

- Compaction is a process in which the free space is collected in a large memory chunk to make some space available for processes.
- In memory management, swapping creates multiple fragments in the memory because of the processes moving in and out.
- Compaction refers to combining all the empty spaces together and processes.
- Compaction helps to solve the problem of fragmentation, but it requires too much of CPU time.
- It moves all the occupied areas of store to one end and leaves one large free space for incoming jobs, instead of numerous small ones.
- In compaction, the system also maintains relocation information and it must be performed on each new allocation of job to the memory or completion of job from memory.

Define compactions.

- - Compaction is a process in which the free space is collected in a large memory chunk to make some space available for processes.
- In memory management, swapping creates multiple fragments in the memory because of the processes moving in and out.
- Compaction refers to combining all the empty spaces together and processes.
- Compaction helps to solve the problem of fragmentation, but it requires too much of CPU time.
- It moves all the occupied areas of store to one end and leaves one large free space for incoming jobs, instead of numerous small ones.
- In compaction, the system also maintains relocation information and it must be performed on each new allocation of job to the memory or completion of job from memory.

Coalescing:

- ❖ **Coalescing** is the act of merging two adjacent free blocks of memory.
- ❖ When an application frees memory, gaps can fall in the memory segment that the application uses.
- ❖ Among other techniques, coalescing is used to reduce external fragmentation, but is not totally effective.
- ❖ Coalescing can be done as soon as blocks are freed, or it can be deferred until sometime later (known as deferred coalescing), or it might not be done at all.

- **What is Virtual Memory?**

- ❖ Virtual Memory is a space where large programs can store themselves in form of pages while their execution and only the required pages or portions of processes are loaded into the main memory.
- ❖ This technique is useful as large virtual memory is provided for user programs when a very small physical memory is there.

Benefits of having Virtual Memory

- Large programs can be written, as virtual space available is huge compared to physical memory.
- Less I/O required, leads to faster and easy swapping of processes.
- More physical memory available, as programs are stored on virtual memory, so they occupy very less space on actual physical memory.
-

Disadvantages:

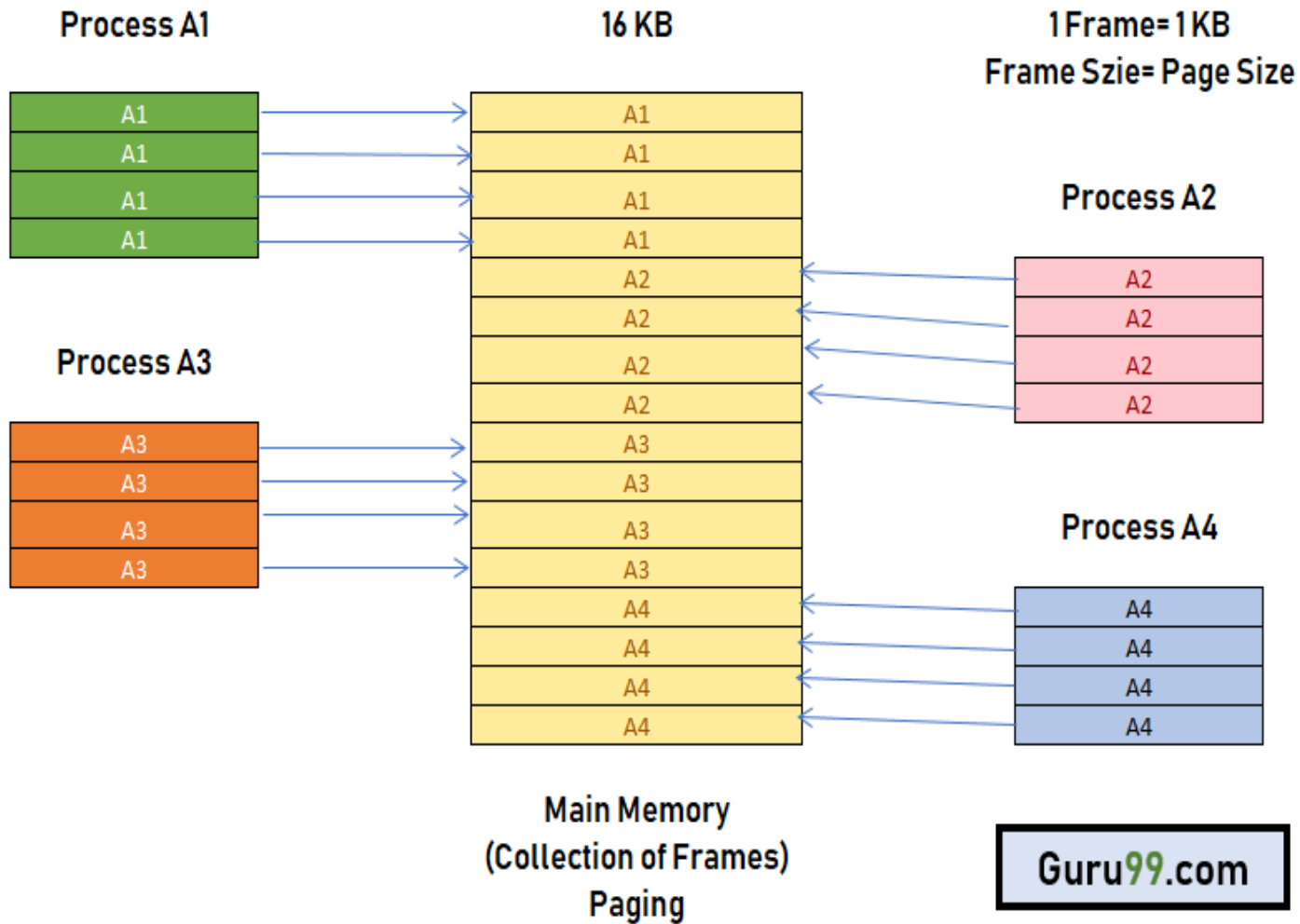
- . Memory requirements.
 - . Longer memory access times.
- This can be improved using TLB

- **What is paging?**

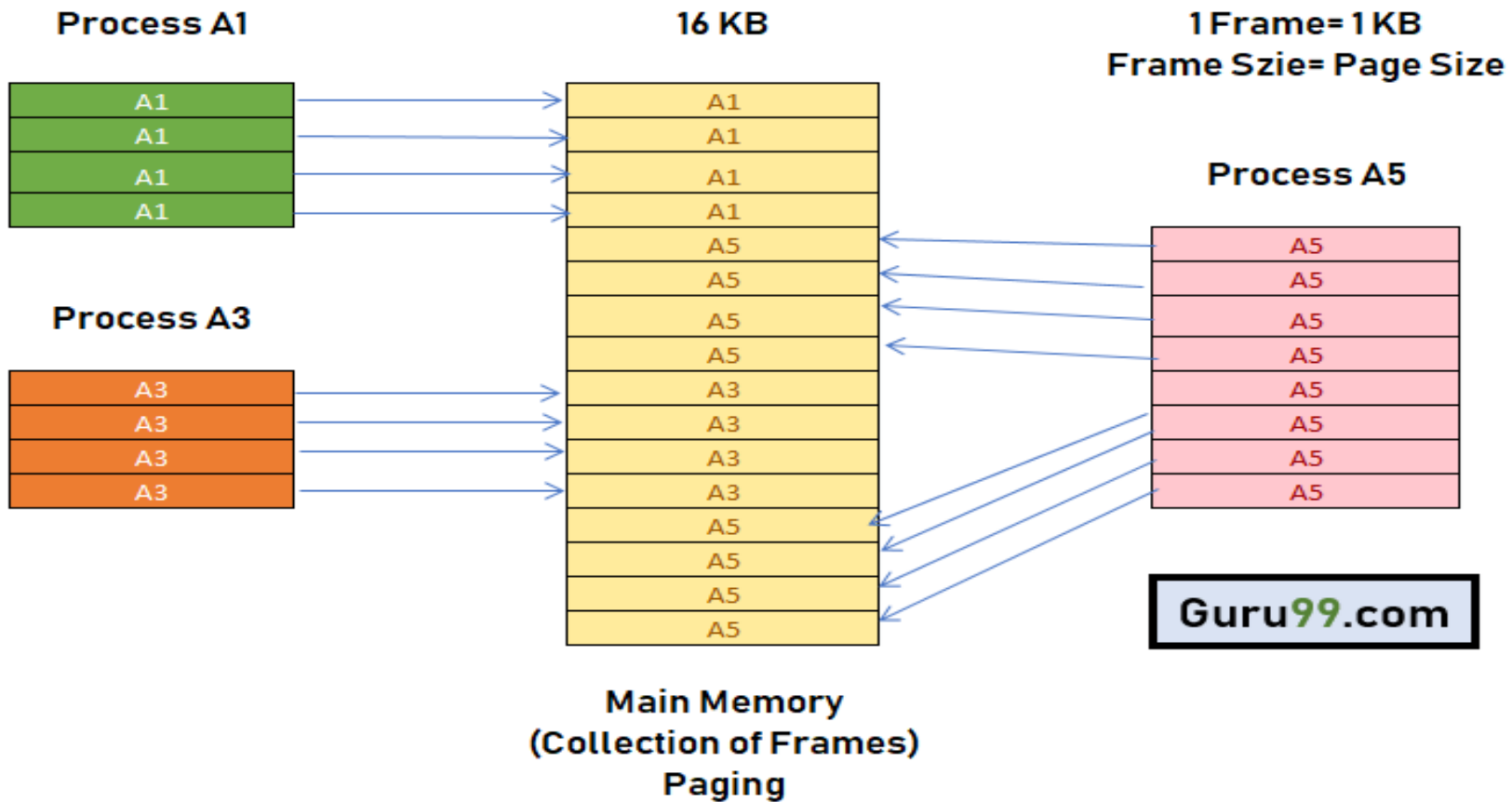
- ❖ **Paging** is a storage mechanism that allows OS to retrieve processes from the secondary storage into the main memory in the form of pages.
- ❖ In the Paging method, the main memory is divided into small fixed-size blocks of physical memory, which is called frames.
- ❖ The size of a frame should be kept the same as that of a page to have maximum utilization of the main memory and to avoid external fragmentation.
- ❖ Paging is used for faster access to data, and it is a logical concept.

Example

- ❖ For example, if the main memory size is 16 KB and Frame size is 1 KB.
- ❖ Here, the main memory will be divided into the collection of 16 frames of 1 KB each.
- ❖ There are 4 separate processes in the system that is A1, A2, A3, and A4 of 4 KB each.
- ❖ Here, all the processes are divided into pages of 1 KB each so that operating system can store one page in one frame.
- ❖ At the beginning of the process, all the frames remain empty so that all the pages of the processes will get stored in a contiguous way.



- ❖ In this example you can see that A2 and A4 are moved to the waiting state after some time.
- ❖ Therefore, eight frames become empty, and so other pages can be loaded in that empty blocks.
- ❖ The process A5 of size 8 pages (8 KB) are waiting in the ready queue.



- ❖ In this example, you can see that there are eight non-contiguous frames which is available in the memory, and paging offers the flexibility of storing the process at the different places.
- ❖ This allows us to load the pages of process A5 instead of A2 and A4.

What is Paging Protection?

- ❖ The paging process should be protected by using the concept of insertion of an additional bit called Valid/Invalid bit.
- ❖ Paging Memory protection in paging is achieved by associating protection bits with each page.
- ❖ These bits are associated with each page table entry and specify protection on the corresponding page.

Advantages of Paging

- Easy to use memory management algorithm
- No need for external Fragmentation
- Swapping is easy between equal-sized pages and page frames.

Disadvantages of Paging

- May cause Internal fragmentation
- Complex memory management algorithm
- Page tables consume additional memory.
- Multi-level paging may lead to memory reference overhead.

Page Table

- ❖ Page Table is a data structure used by the virtual memory system to store the mapping between logical addresses and physical addresses.
- ❖ Logical addresses are generated by the CPU for the pages of the processes therefore they are generally used by the processes.
- ❖ Physical addresses are the actual frame address of the memory.
- ❖ They are generally used by the hardware or more specifically by RAM subsystems.

The image given below considers,

Physical Address Space = M words

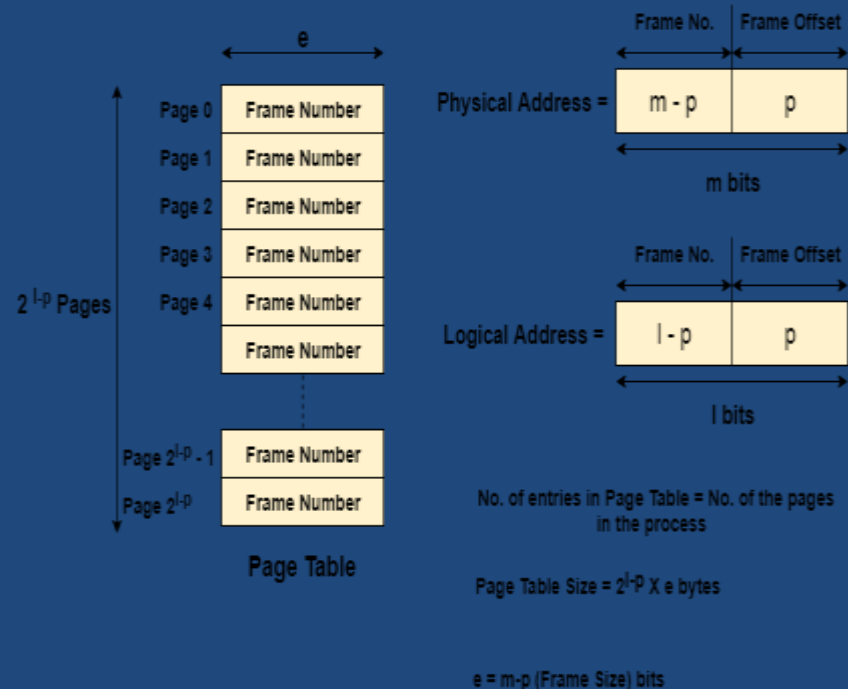
Logical Address Space = L words

Page Size = P words

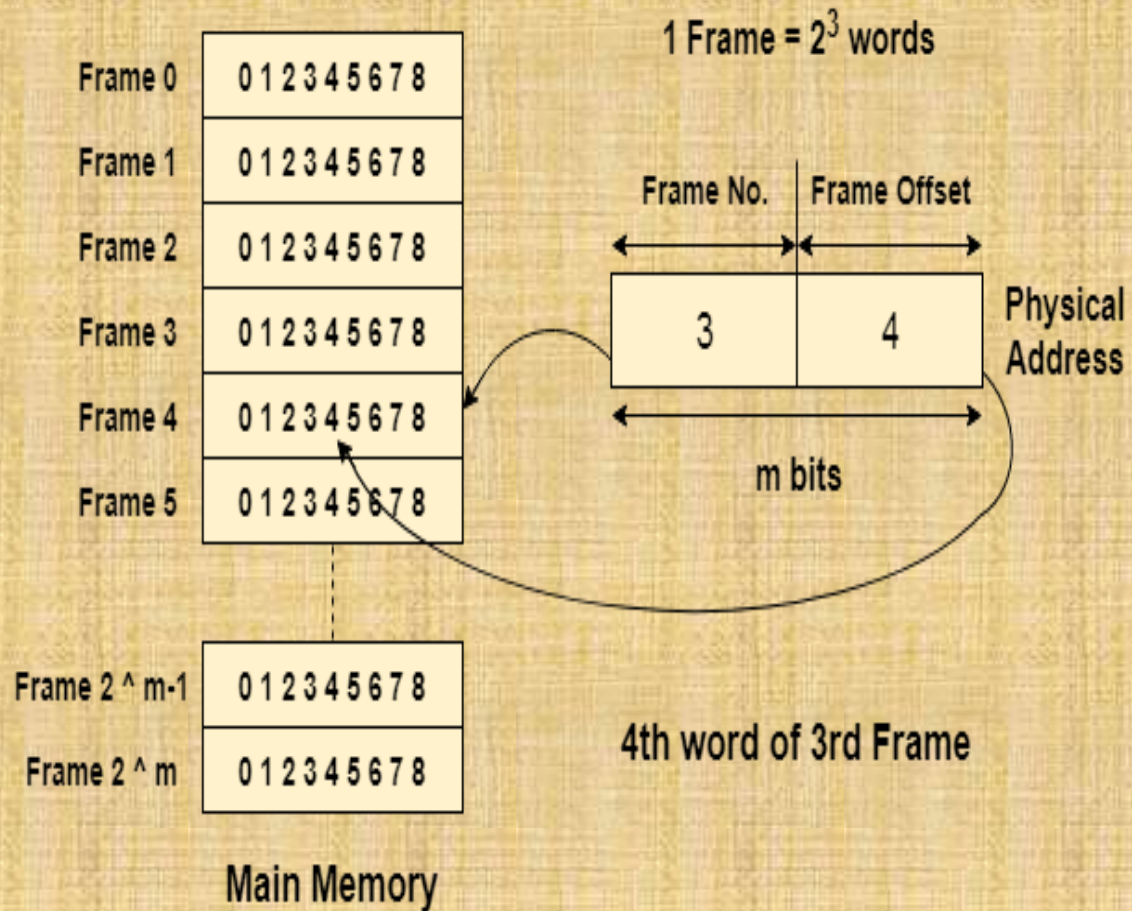
Physical Address = $\log_2 M = m$ bits

Logical Address = $\log_2 L = l$ bits

page offset = $\log_2 P = p$ bits



- ❖ The CPU always accesses the processes through their logical addresses. However, the main memory recognizes physical address only.
- ❖ In this situation, a unit named as Memory Management Unit comes into the picture.
- ❖ It converts the page number of the logical address to the frame number of the physical address.
- ❖ The offset remains same in both the addresses.
- ❖ To perform this task, Memory Management unit needs a special kind of mapping which is done by page table.
- ❖ The page table stores all the Frame numbers corresponding to the page numbers of the page table.
- ❖ In other words, the page table maps the page number to its actual location (frame number) in the memory.
- ❖ In the image given below shows, how the required word of the frame is accessed with the help of offset.



❖ Page Table-

- Page table is a data structure.
- It maps the page number referenced by the CPU to the frame number where that page is stored.

❖ Characteristics-

- Page table is stored in the main memory.
- Number of entries in a page table = Number of pages in which the process is divided.
- Page Table Base Register (PTBR) contains the base address of page table.
- Each process has its own independent page table.

Page Fault :

- ❖ A page fault occurs when a program attempts to access a block of memory that is not stored in the physical memory, or RAM.
- ❖ The fault notifies the operating system that it must locate the data in virtual memory, and then transfer it from the storage device, such as an HDD or SSD, to the system RAM.

Page Replacement Algorithms

- **Page Fault** – A page fault happens when a running program accesses a memory page that is mapped into the virtual address space, but not loaded in physical memory.

Page Replacement Algorithms :

- ❖ **First In First Out (FIFO)** –

This is the simplest page replacement algorithm.

- ❖ In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue.
- ❖ When a page needs to be replaced page in the front of the queue is selected for removal.
- **Example-1** Consider page reference string 1, 3, 0, 3, 5, 6 with 3 page frames. Find number of page faults.

Page
reference

1, 3, 0, 3, 5, 6, 3

1	3	0	3	5	6	3
		0	0	0	0	3
	3	3	3	3	6	6
1	1	1	1	5	5	5
Miss	Miss	Miss	Hit	Miss	Miss	Miss

Total Page Fault = 6

- Initially all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots —> **3 Page Faults.**
when 3 comes, it is already in memory so —> **0 Page Faults.**

Then 5 comes, it is not available in memory so it replaces the oldest page slot i.e 1. —> **1 Page Fault.**

6 comes, it is also not available in memory so it replaces the oldest page slot i.e 3 —> **1 Page Fault.**

Finally when 3 come it is not available so it replaces 0 **1 page fault**

Optimal Page replacement –

- ❖ In this algorithm, pages are replaced which would not be used for the longest duration of time in the future.
- **Example-2:** Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, with 4 page frame. Find number of page fault.

Page
reference

7,0,1,2,0,3,0,4,2,3,0,3,2,3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total Page Fault = 6

Initially all slots are empty, so when 7 0 1 2 are allocated to the empty slots

—> **4 Page faults**

0 is already there so —> **0 Page fault.**

when 3 came it will take the place of 7 because it is not used for the longest duration of time in the future.—>**1 Page fault.**

0 is already there so —> **0 Page fault..**

4 will takes place of 1 —> **1 Page Fault.**

Now for the further page reference string —> **0 Page fault** because they are already available in the memory.

Optimal page replacement is perfect, but not possible in practice as the operating system cannot know future requests. The use of Optimal Page replacement is to set up a benchmark so that other replacement algorithms can be analyzed against it.

1. First In First Out (FIFO):

- . Advantages –
 - It is simple and easy to understand & implement.
- . Disadvantages –
 - The process effectiveness is low.
 - When we increase the number of frames while using FIFO, we are giving more memory to processes. So, page fault should decrease, but here the page faults are increasing. This problem is called as Belady's Anomaly.
 - Every frame needs to be taken account off.

- Optimal Page Replacement (OPR):

- . Advantages –

- Complexity is less and easy to implement.
- Assistance needed is low i.e Data Structure used are easy and light.

- . Disadvantages –

- OPR is perfect, but not possible in practice as the operating system cannot know future requests.
- Error handling is tough.

Segmentation

- ❖ Segmentation is a memory management technique in which, the memory is divided into the variable size parts. Each part is known as segment which can be allocated to a process.
- ❖ The details about each segment are stored in a table called as segment table. Segment table is stored in one (or many) of the segments.
- ❖ Segment table contains mainly two information about segment:
 - Base: It is the base address of the segment
 - Limit: It is the length of the segment.

- **Why Segmentation is required?**

- ❖ Operating system doesn't care about the User's view of the process.
- ❖ It may divide the same function into different pages and those pages may or may not be loaded at the same time into the memory.
- ❖ It decreases the efficiency of the system.
- ❖ It is better to have segmentation which divides the process into the segments.
- ❖ Each segment contain same type of functions such as main function can be included in one segment and the library functions can be included in the other segment,

Advantages of Segmentation

- No internal fragmentation
- Average Segment Size is larger than the actual page size.
- Less overhead
- It is easier to relocate segments than entire address space.
- The segment table is of lesser size as compare to the page table in paging.

Disadvantages

- It can have external fragmentation.
- it is difficult to allocate contiguous memory to variable sized partition.
- Costly memory management algorithms.

Paging VS Segmentation

Sr No.	Paging	Segmentation
1	Non-Contiguous memory allocation	Non-contiguous memory allocation
2	Paging divides program into fixed size pages.	Segmentation divides program into variable size segments.
3	OS is responsible	Compiler is responsible.
4	Paging is faster than segmentation	Segmentation is slower than paging
5	Paging is closer to Operating System	Segmentation is closer to User
6	It suffers from internal fragmentation	It suffers from external fragmentation
7	There is no external fragmentation	There is no external fragmentation
8	Logical address is divided into page number and page offset	Logical address is divided into segment number and segment offset
9	Page table is used to maintain the page information.	Segment Table maintains the segment information
10	Page table entry has the frame number and some flag bits to represent details about pages.	Segment table entry has the base address of the segment and some protection bits for the segments.

Disadvantages:

- . External fragmentation.
- . Costly memory management algorithms.
- . Segmentation: find free memory area big enough.
- . Paging: keep list of free pages, any page is ok.
- Segments of unequal size not suited as well for swapping

Difference between Paging and Segmentation

Paging

A page is of the fixed block size.

It may lead to internal fragmentation.

In Paging, the hardware decides the page size.

A process address space is broken into fixed-sized blocks, which is called pages.

The paging technique is faster for memory access.

Page table stores the page data

Paging does not facilitate any sharing of procedures.

Paging fails to distinguish and secure procedures and data separately.

Paging address space is one dimensional

In paging, the user just provides a single integer as the address, that is divided by the hardware into a page number and offset.

Segmentation

A segment is of variable size.

It may lead to external fragmentation.

The segment size is specified by the user.

A process address space is broken in differing sized blocks called sections.

Segmentation is slower than paging method.

Segmentation table stores the segmentation data.

Segmentation allows for the sharing of procedures.

Segmentation can be able to separate secure procedures and data.

In segmentation, there is the availability of many independent address spaces

In the segmentation method, the user specifies the address in two quantities
1)segment number 2)offset.

Advantage of Segmentation

- Simple to relocate segments than the entire address space.
- The absence of internal fragmentation as external fragmentation has to be done.
- The segment table is of lesser size compared with the page table in paging.
- The average size of the segment is larger to the actual size of the page
- Offer protection within the segment
- Not offers internal fragmentation
- Segment tables use lesser memory than paging
- As it offers the small segment table, memory reference is simple, which lends itself to sharing data among processes.

Disadvantages of Paging

- Paging causes internal fragmentation on older systems.
- Longer memory lookup times compared to segmentation
- It may cause internal fragmentation
- Complex memory management algorithm
- Page tables consume additional memory.
- Multi-level paging may lead to memory reference overhead.